OpenClaw Architecture

Technical Whitepaper: Multi-Channel AI Agent Gateway

OpenClaw Project
Community-Driven Open Source

February 19, 2026
Version 1.0

"EXFOLIATE! EXFOLIATE!" — A space lobster, probably

Contents

1 Executive Summary **4**
1.1 Key Features 4

2 System Architecture **4**
2.1 High-Level Architecture 4
2.2 Component Breakdown 5
2.2.1 Gateway (Daemon) 5
2.2.2 Clients 5
2.2.3 Nodes 5
2.2.4 Agents 6

3 WebSocket Protocol **6**
3.1 Connection Lifecycle 6
3.2 Frame Format 6
3.3 Idempotency 7

4 Session Management **8**
4.1 Session Scoping 8
4.2 Session Reset Policy 8
4.3 Session Pruning 9

5 Multi-Agent Routing **9**
5.1 Routing Algorithm 9
5.2 Binding Example 10

6 Model Management **10**
6.1 Model Selection Hierarchy 10
6.2 Model Catalog Format 11
6.3 Cost Estimation 11

7 Security Architecture **12**
7.1 Authentication Layers 12
7.2 Pairing Protocol 12
7.3 DM Access Control 12

8 Configuration Management **13**
8.1 Configuration Hot Reload 13
8.2 Configuration Validation 13
8.3 Environment Variable Substitution 14

9 Automation **14**
9.1 Cron Jobs 14
9.2 Webhooks 14

10 Performance Optimization **15**
10.1 Context Pruning 15
10.2 Session Compaction 15
10.3 Token Budget Management 15

11 Extensibility **16**
11.1 Plugin Architecture 16
11.2 Skill System 16

12 Deployment Patterns **17**
12.1 Single-Host Deployment 17
12.2 Remote Access Deployment 17
12.3 Multi-Agent Deployment 17

13 Message Flow **17**
13.1 Inbound Message Processing 17
13.2 Tool Execution Flow 17

14 Monitoring & Observability **18**
14.1 Health Checks 18
14.2 Metrics Collection 18
14.3 Logging 19

15 Conclusion **19**

16 References **20**

A Configuration Examples **20**
A.1 Minimal Configuration 20
A.2 Multi-Agent Configuration 20
A.3 Secure Multi-User Configuration 21

B API Reference **22**
B.1 WebSocket Methods 22
B.2 Event Types 22

1 Executive Summary

OpenClaw is a self-hosted, multi-channel gateway that connects messaging platforms to AI coding agents. It provides a unified interface for interacting with AI assistants across WhatsApp, Telegram, Discord, iMessage, and other platforms, while maintaining user privacy and data sovereignty.

1.1 Key Features

- **Self-Hosted**: Runs on user's hardware, ensuring complete data control
- **Multi-Channel**: Simultaneous support for multiple messaging platforms
- **Agent-Native**: Built for coding agents with tool use, sessions, and memory
- **Multi-Agent Routing**: Isolated agents with workspace segregation
- **Open Source**: MIT licensed, community-driven development
- **WebSocket Protocol**: Real-time bidirectional communication

2 System Architecture

2.1 High-Level Architecture

OpenClaw follows a hub-and-spoke architecture where a central Gateway daemon manages all connections and routing.

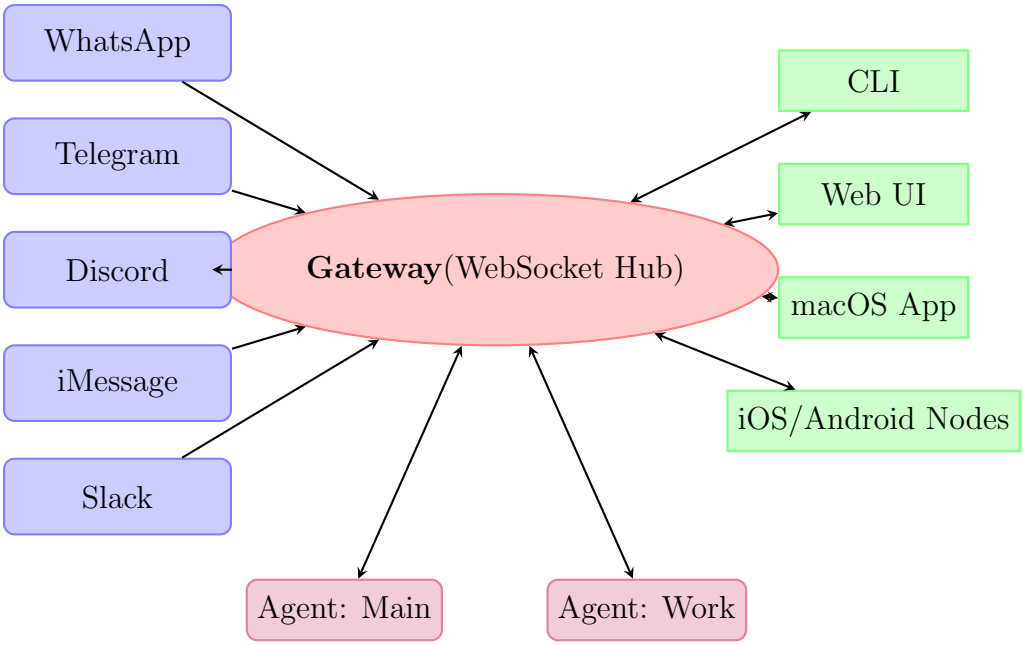

Figure 1: OpenClaw High-Level Architecture

2.2 Component Breakdown

2.2.1 Gateway (Daemon)

The Gateway is the central orchestrator responsible for:

1. **Provider Management**: Maintains persistent connections to messaging platforms (Baileys for WhatsApp, grammY for Telegram, etc.)
2. **WebSocket API**: Exposes typed WebSocket interface for clients and nodes
3. **Request Validation**: Validates inbound frames against JSON Schema
4. **Event Broadcasting**: Emits events (`agent`, `chat`, `presence`, `health`, `heartbeat`, `cron`)
5. **Session Management**: Routes messages to appropriate agents based on bindings

Connection Details:

- Default bind: `127.0.0.1:18789`
- Protocol: WebSocket over TCP
- Authentication: Token-based (optional `OPENCLAW_GATEWAY_TOKEN`)
- Transport: Text frames with JSON payloads

2.2.2 Clients

Clients connect to the Gateway via WebSocket to send commands and receive events:

- **CLI**: Command-line interface for configuration and control
- **Web Control UI**: Browser-based dashboard (`http://127.0.0.1:18789`)
- **macOS App**: Native desktop application
- **WebChat**: Static web interface for chat history

2.2.3 Nodes

Nodes are paired devices (iOS/Android/macOS/headless) that:

- Connect with `role: node` identifier
- Provide device identity for pairing approval
- Expose commands: `canvas.*`, `camera.*`, `screen.record`, `location.get`
- Support Canvas rendering and A2UI (Agent-to-UI)

2.2.4 Agents

An agent is a fully isolated AI "brain" with:

- **Workspace**: Files, persona rules (AGENTS.md, SOUL.md, USER.md)
- **State Directory**: Auth profiles, model registry, per-agent config
- **Session Store**: Chat history and routing state

Path Structure:

```
~/.openclaw/
          openclaw.json                # Main config
          workspace/                   # Default workspace
          agents/
                main/
                      agent/             # Auth profiles
                      sessions/          # Session store
                work/
                    agent/
                    sessions/
```

3 WebSocket Protocol

3.1 Connection Lifecycle

3.2 Frame Format

OpenClaw uses JSON-based message framing:

Request Frame

```
{
  "type": "req",
  "id": "<uuid>",
  "method": "agent|send|health|...",
  "params": { ... }
}
```

Response Frame

```
{
  "type": "res",
  "id": "<uuid>",
  "ok": true,
  "payload": { ... }  // or "error": { ... }
}
```

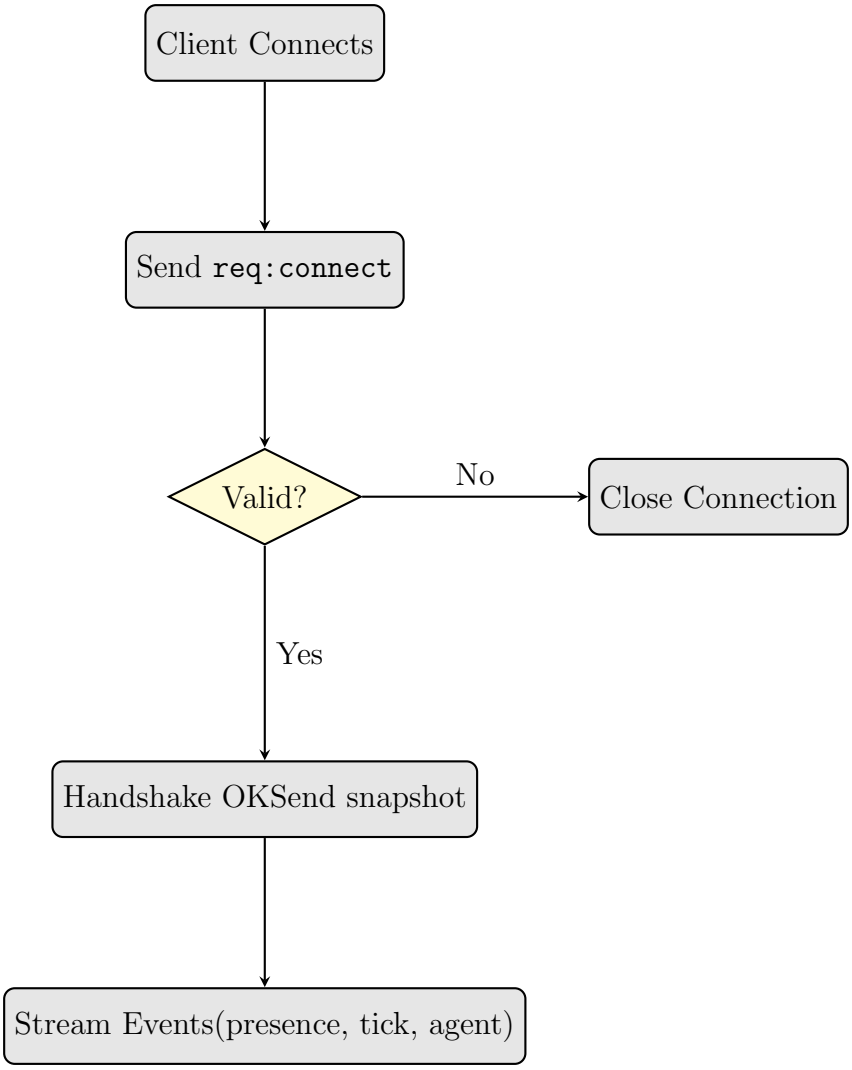

Figure 2: WebSocket Connection Lifecycle

Event Frame

```
{
  "type": "event",
  "event": "agent|chat|presence|...",
  "payload": { ... },
  "seq": 123,
  "stateVersion": 456
}
```

3.3 Idempotency

Side-effecting methods (`send`, `agent`) require idempotency keys to safely retry. The Gateway maintains a short-lived deduplication cache.

Formula for idempotency window:

$$T_{expire} = T_{request} + \Delta T_{cache}$$

where ΔT_{cache} is typically 5 minutes.

4 Session Management

4.1 Session Scoping

Sessions control conversation continuity and isolation. The session key structure follows:

$$\text{SessionKey} = \text{agent} : \langle\text{agentId}\rangle : \langle\text{scope}\rangle$$

DM Scoping Modes (`session.dmScope`):

1. `main`: All DMs share one session

$$\text{SessionKey} = \text{agent} : \langle\text{agentId}\rangle : \text{main}$$

2. `per-peer`: Isolate by sender

$$\text{SessionKey} = \text{agent} : \langle\text{agentId}\rangle : \text{dm} : \langle\text{peerId}\rangle$$

3. `per-channel-peer`: Isolate by channel + sender (recommended)

$$\text{SessionKey} = \text{agent} : \langle\text{agentId}\rangle : \langle\text{channel}\rangle : \text{dm} : \langle\text{peerId}\rangle$$

4. `per-account-channel-peer`: Isolate by account + channel + sender

$$\text{SessionKey} = \text{agent} : \langle\text{agentId}\rangle : \langle\text{channel}\rangle : \langle\text{accountId}\rangle : \text{dm} : \langle\text{peerId}\rangle$$

4.2 Session Reset Policy

Sessions are reused until they expire. Expiry is evaluated on the next inbound message.

Daily Reset Formula:

$$\text{isStale} = T_{lastUpdate} < T_{lastDailyReset}$$

where $T_{lastDailyReset}$ defaults to 4:00 AM local time on the Gateway host.

Idle Reset Formula:

$$\text{isIdle} = (T_{now} - T_{lastUpdate}) > \Delta T_{idle}$$

Combined Reset (whichever expires first):

$$\text{shouldReset} = \text{isStale} \vee \text{isIdle}$$

4.3 Session Pruning

OpenClaw trims old tool results from in-memory context before LLM calls:

Context Window Management:

$$C_{effective} = C_{total} - C_{pruned}$$

where:

- C_{total} = total context tokens
- C_{pruned} = tokens from pruned tool results
- $C_{effective}$ = effective context sent to LLM

This does not rewrite JSONL history, only affects in-memory context.

5 Multi-Agent Routing

5.1 Routing Algorithm

Bindings are deterministic with a most-specific-wins policy:

1. `peer` match (exact DM/group/channel ID)
2. `parentPeer` match (thread inheritance)
3. `guildId + roles` (Discord role routing)
4. `guildId` (Discord)
5. `teamId` (Slack)
6. `accountId` match for a channel
7. channel-level match (`accountId: "*"`)
8. fallback to default agent

Routing Decision Function:

$$\text{agent}(\text{msg}) = \begin{cases} \text{agent}_{\text{peer}} & \text{if } \exists \text{ peer match} \\ \text{agent}_{\text{parent}} & \text{if } \exists \text{ parent match} \\ \text{agent}_{\text{guild}} & \text{if } \exists \text{ guild match} \\ \text{agent}_{\text{channel}} & \text{if } \exists \text{ channel match} \\ \text{agent}_{\text{default}} & \text{otherwise} \end{cases}$$

5.2 Binding Example

```
{
  "agents": {
    "list": [
      {
        "id": "home",
        "default": true,
        "workspace": "~/.openclaw/workspace-home"
      },
      {
        "id": "work",
        "workspace": "~/.openclaw/workspace-work"
      }
    ]
  },
  "bindings": [
    {
      "agentId": "home",
      "match": {
        "channel": "whatsapp",
        "accountId": "personal"
      }
    },
    {
      "agentId": "work",
      "match": {
        "channel": "whatsapp",
        "accountId": "biz"
      }
    }
  ]
}
```

6 Model Management

6.1 Model Selection Hierarchy

OpenClaw selects models in this order:

1. **Primary** model: `agents.defaults.model.primary`
2. **Fallbacks**: `agents.defaults.model.fallbacks` (in order)
3. **Provider auth failover**: Inside a provider before moving to next model

Model Selection Function:

$$M_{selected} = \begin{cases} M_{primary} & \text{if available} \\ M_{fallback}[0] & \text{if } M_{primary} \text{ fails} \\ M_{fallback}[1] & \text{if } M_{fallback}[0] \text{ fails} \\ \vdots & \\ M_{fallback}[n] & \text{if all previous fail} \end{cases}$$

6.2 Model Catalog Format

```
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "anthropic/claude-sonnet-4-5",
        "fallbacks": ["openai/gpt-5.2"]
      },
      "models": {
        "anthropic/claude-sonnet-4-5": {
          "alias": "Sonnet"
        },
        "openai/gpt-5.2": {
          "alias": "GPT"
        }
      }
    }
  }
}
```

6.3 Cost Estimation

Token cost can be estimated using:

$$\text{Cost} = \left(\frac{T_{input}}{1000} \times P_{input}\right) + \left(\frac{T_{output}}{1000} \times P_{output}\right)$$

where:

- T_{input} = input tokens
- T_{output} = output tokens
- P_{input} = price per 1K input tokens
- P_{output} = price per 1K output tokens

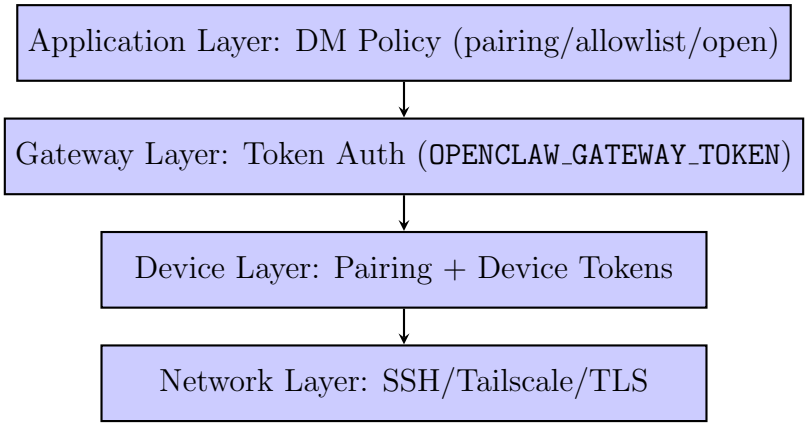

Figure 3: Security Layers

7 Security Architecture

7.1 Authentication Layers

7.2 Pairing Protocol

Device Trust Flow:

1. Client sends `connect` with device identity
2. Gateway generates challenge nonce: $N_{challenge}$
3. Client signs challenge: $S = \text{Sign}(N_{challenge}, K_{device})$
4. Gateway validates signature
5. On approval: Gateway issues device token T_{device}
6. Future connects use T_{device} for authentication

Local vs Remote Pairing:

$$\text{AutoApprove} = \begin{cases} \text{true} & \text{if IsLocal(IP)} \vee \text{IsTailnet(IP)} \\ \text{false} & \text{otherwise (requires explicit approval)} \end{cases}$$

7.3 DM Access Control

DM Policy Modes:

- `pairing`: Unknown senders get one-time pairing code
- `allowlist`: Only senders in `allowFrom`
- `open`: Allow all (requires `allowFrom: ["*"]`)
- `disabled`: Ignore all DMs

Access Decision Function:

$$\text{Allow}(\text{sender}) = \begin{cases} \text{true} & \text{if policy} = \text{open} \\ \text{sender} \in \text{allowFrom} & \text{if policy} = \text{allowlist} \\ \text{IsPaired}(\text{sender}) & \text{if policy} = \text{pairing} \\ \text{false} & \text{if policy} = \text{disabled} \end{cases}$$

8 Configuration Management

8.1 Configuration Hot Reload

OpenClaw watches `~/.openclaw/openclaw.json` and applies changes automatically.

Reload Modes:

- `hybrid` (default): Hot-apply safe changes, auto-restart for critical ones
- `hot`: Hot-apply safe changes only, warn when restart needed
- `restart`: Restart Gateway on any config change
- `off`: Disable file watching

Hot-Apply Categories:

Category	Restart Needed?
Channels	No
Agents & Models	No
Automation (hooks, cron)	No
Sessions & Messages	No
Tools & Media	No
UI & Logging	No
Gateway Server (port, bind, auth)	**Yes**
Infrastructure (discovery, plugins)	**Yes**

Table 1: Configuration Hot-Apply Matrix

8.2 Configuration Validation

OpenClaw enforces strict JSON Schema validation:

Validation Function:

$$\text{Valid}(\text{config}) = \text{Schema.validate}(\text{config}) \wedge \text{NoUnknownKeys}(\text{config})$$

If validation fails:

1. Gateway refuses to start
2. Only diagnostic commands work (`openclaw doctor`, `openclaw health`)
3. Run `openclaw doctor --fix` to apply repairs

8.3 Environment Variable Substitution

Config values support env var substitution:

```
{
  "gateway": {
    "auth": {
      "token": "${OPENCLAW_GATEWAY_TOKEN}"
    }
  }
}
```

Substitution Rules:

- Only uppercase names: `[A-Z_][A-Z0-9_]*`
- Missing/empty vars throw error at load time
- Escape with `$${VAR}` for literal output
- Works inside `$include` files

9 Automation

9.1 Cron Jobs

OpenClaw supports scheduled tasks via cron jobs:

```
{
  "cron": {
    "enabled": true,
    "maxConcurrentRuns": 2,
    "sessionRetention": "24h"
  }
}
```

Job Types:

1. **systemEvent**: Injects text as system event into session
2. **agentTurn**: Runs agent with message (isolated sessions only)

9.2 Webhooks

HTTP webhook endpoints on the Gateway:

```
{
  "hooks": {
    "enabled": true,
    "token": "shared-secret",
    "path": "/hooks",
    "defaultSessionKey": "hook:ingress",
    "mappings": [
      {
```

```
        "match": { "path": "gmail" },
        "action": "agent",
        "agentId": "main",
        "deliver": true
      }
    ]
  }
}
```

10 Performance Optimization

10.1 Context Pruning

OpenClaw implements aggressive context pruning to manage token costs:

Pruning Strategy:

$$\text{Keep}(\text{message}) = \begin{cases} \text{true} & \text{if type} \in \{\text{user}, \text{assistant}\} \\ \text{true} & \text{if age} < \Delta T_{recent} \\ \text{false} & \text{if type} = \text{tool_result} \wedge \text{age} > \Delta T_{stale} \end{cases}$$

where ΔT_{recent} is typically 5 turns and ΔT_{stale} is 10 turns.

10.2 Session Compaction

Pre-compaction memory flush runs when session nears auto-compaction:

Compaction Trigger:

$$\text{TriggerCompaction} = \frac{C_{current}}{C_{max}} > \theta_{compact}$$

where $\theta_{compact}$ is typically 0.8 (80% of context window).

10.3 Token Budget Management

Token Allocation Formula:

$$B_{available} = C_{max} - C_{system} - C_{workspace} - C_{history}$$

where:

- C_{max} = model's maximum context window
- C_{system} = system prompt tokens
- $C_{workspace}$ = injected workspace file tokens
- $C_{history}$ = conversation history tokens

11 Extensibility

11.1 Plugin Architecture

OpenClaw supports channel plugins for extensibility:

```
{
  "plugins": {
    "channels": [
      "openclaw-channel-mattermost",
      "openclaw-channel-slack-enterprise"
    ]
  }
}
```

Plugin Loading:

1. Gateway scans `node_modules` for `openclaw-channel-*`
2. Validates plugin manifest against schema
3. Registers channel handlers
4. Merges plugin config into main config

11.2 Skill System

Skills extend agent capabilities:

```
~/.openclaw/workspace/
          skills/
                weather/
                      SKILL.md
                      scripts/
                github/
                    SKILL.md
                    scripts/
```

Skill Discovery:

$$\text{Skills} = \text{Workspace Skills} \cup \text{Global Skills}$$

where:

- Workspace Skills: `<workspace>/skills/`
- Global Skills: `~/.openclaw/skills/`

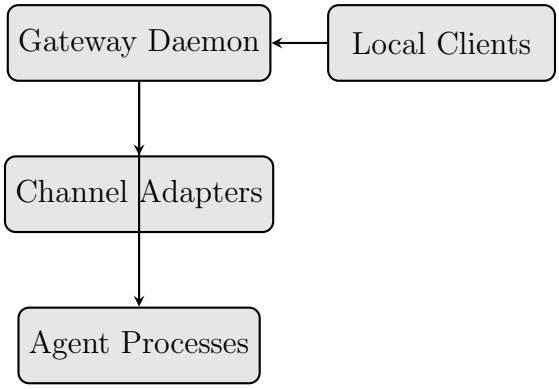

Figure 4: Single-Host Deployment

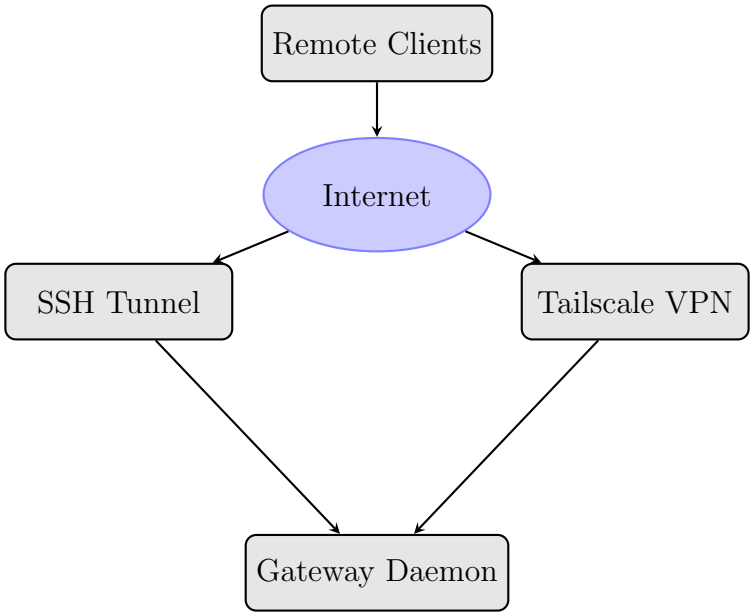

Figure 5: Remote Access Deployment

12 Deployment Patterns

12.1 Single-Host Deployment

12.2 Remote Access Deployment

12.3 Multi-Agent Deployment

13 Message Flow

13.1 Inbound Message Processing

13.2 Tool Execution Flow

1. LLM generates tool call in response
2. Gateway validates tool against allowlist

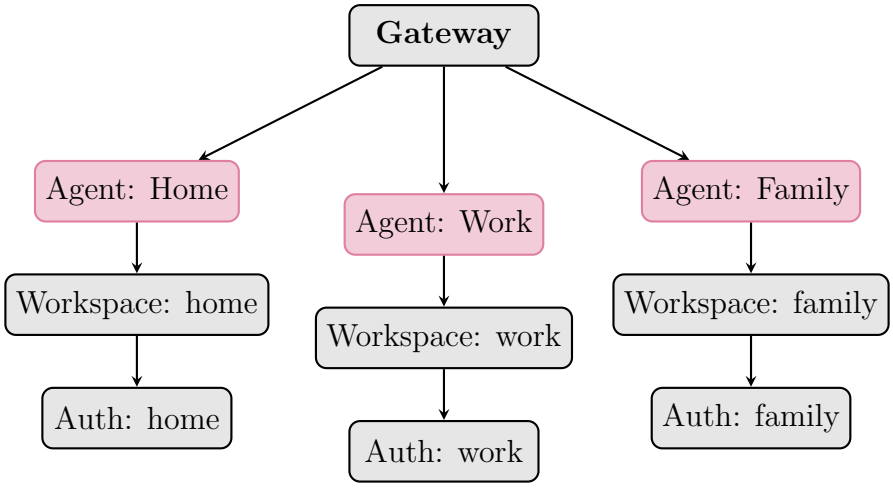

Figure 6: Multi-Agent Deployment

3. Tool executes (local or sandboxed)
4. Result appended to conversation
5. LLM called again with tool result
6. Process repeats until no more tool calls

14 Monitoring & Observability

14.1 Health Checks

Health Status Function:

$$H = \begin{cases} \text{healthy} & \text{if all channels connected} \wedge \text{agents responsive} \\ \text{degraded} & \text{if some channels disconnected} \\ \text{unhealthy} & \text{if no channels connected} \vee \text{agents unresponsive} \end{cases}$$

14.2 Metrics Collection

OpenClaw tracks:

- **Message throughput**: Messages per second per channel
- **Token usage**: Input/output tokens per session
- **API costs**: Estimated cost per conversation
- **Latency**: Time from message receipt to response delivery
- **Error rates**: Failed messages, auth errors, tool failures

Latency Calculation:

$$L_{total} = L_{queue} + L_{llm} + L_{tools} + L_{send}$$

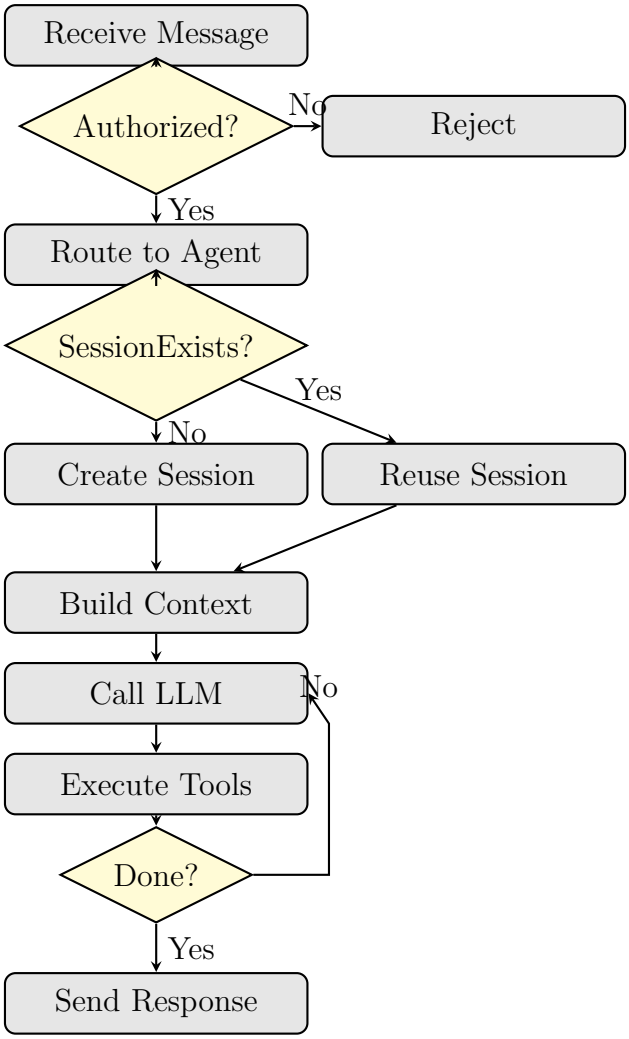

Figure 7: Inbound Message Processing Flow

14.3 Logging

Log levels and targets:

- **ERROR**: Critical failures, logged to stderr
- **WARN**: Recoverable issues, logged to stderr
- **INFO**: Operational events, logged to stdout
- **DEBUG**: Detailed traces, opt-in via config

15 Conclusion

OpenClaw provides a robust, extensible architecture for connecting AI agents to multiple messaging platforms. Its key innovations include:

1. **Unified Gateway**: Single point of control for all channels
2. **WebSocket Protocol**: Real-time, bidirectional communication

3. **Multi-Agent Support**: Isolated workspaces and routing
4. **Session Management**: Flexible scoping and reset policies
5. **Security Layers**: Pairing, device tokens, and access control
6. **Hot Reload**: Zero-downtime configuration updates
7. **Extensibility**: Plugin architecture and skill system

The architecture balances ease of use with powerful features, making it suitable for both individual developers and multi-user deployments.

16 References

1. OpenClaw Documentation: `https://docs.openclaw.ai`
2. OpenClaw GitHub: `https://github.com/openclaw/openclaw`
3. OpenClaw Community: `https://discord.com/invite/clawd`
4. ClawHub Skills: `https://clawhub.com`

A Configuration Examples

A.1 Minimal Configuration

```
{
  "agents": {
    "defaults": {
      "workspace": "~/.openclaw/workspace"
    }
  },
  "channels": {
    "whatsapp": {
      "allowFrom": ["+15555550123"]
    }
  }
}
```

A.2 Multi-Agent Configuration

```
{
  "agents": {
    "list": [
      {
        "id": "home",
        "default": true,
        "workspace": "~/.openclaw/workspace-home",
```

```
        "model": "anthropic/claude-sonnet-4-5"
      },
      {
        "id": "work",
        "workspace": "~/.openclaw/workspace-work",
        "model": "anthropic/claude-opus-4-6"
      }
    ]
  },
  "bindings": [
    {
      "agentId": "home",
      "match": {
        "channel": "whatsapp",
        "accountId": "personal"
      }
    },
    {
      "agentId": "work",
      "match": {
        "channel": "telegram"
      }
    }
  ],
  "channels": {
    "whatsapp": {
      "dmPolicy": "allowlist",
      "allowFrom": ["+15555550123"]
    },
    "telegram": {
      "dmPolicy": "pairing",
      "botToken": "${TELEGRAM_BOT_TOKEN}"
    }
  }
}
```

A.3 Secure Multi-User Configuration

```
{
  "session": {
    "dmScope": "per-channel-peer"
  },
  "channels": {
    "whatsapp": {
      "dmPolicy": "allowlist",
      "allowFrom": [
        "+15555550001",
        "+15555550002"
      ]
    }
```

```
  }
}
```

B API Reference

B.1 WebSocket Methods

Method	Description
`connect`	Handshake + authentication
`health`	Gateway health status
`status`	Session store + recent sessions
`send`	Send message to channel
`agent`	Run agent with message
`sessions.list`	List all sessions
`sessions.get`	Get session details
`sessions.delete`	Delete session
`config.get`	Get current config
`config.apply`	Apply full config
`config.patch`	Partial config update
`cron.list`	List cron jobs
`cron.add`	Add cron job
`cron.remove`	Remove cron job

Table 2: WebSocket API Methods

B.2 Event Types

Event	Description
`agent`	Agent response streaming
`chat`	New inbound message
`presence`	Channel presence update
`health`	Gateway health change
`heartbeat`	Periodic heartbeat
`tick`	Clock tick (every second)
`cron`	Cron job execution
`shutdown`	Gateway shutting down

Table 3: WebSocket Event Types